FORWARD DEPLOYED AI ENGINEER · TAKE-HOME BUSINESS CASE

Account Expansion Opportunity Radar

A first-win proposal for Westbrook Talent Partners

Author · André Saraiva **For** · Grandview Tek

Primary KPI · net-new gross margin from expansion within existing accounts

Companion · working repo (`westbrook-radar`)

How to read this. Section 1 is the whole idea in one minute. Sections 2–5 are the four required deliverables. Section 6 is the **working demo** — the centerpiece; run it in one command. The repo is buildable by a peer engineer; this page is readable by an executive in one sitting.

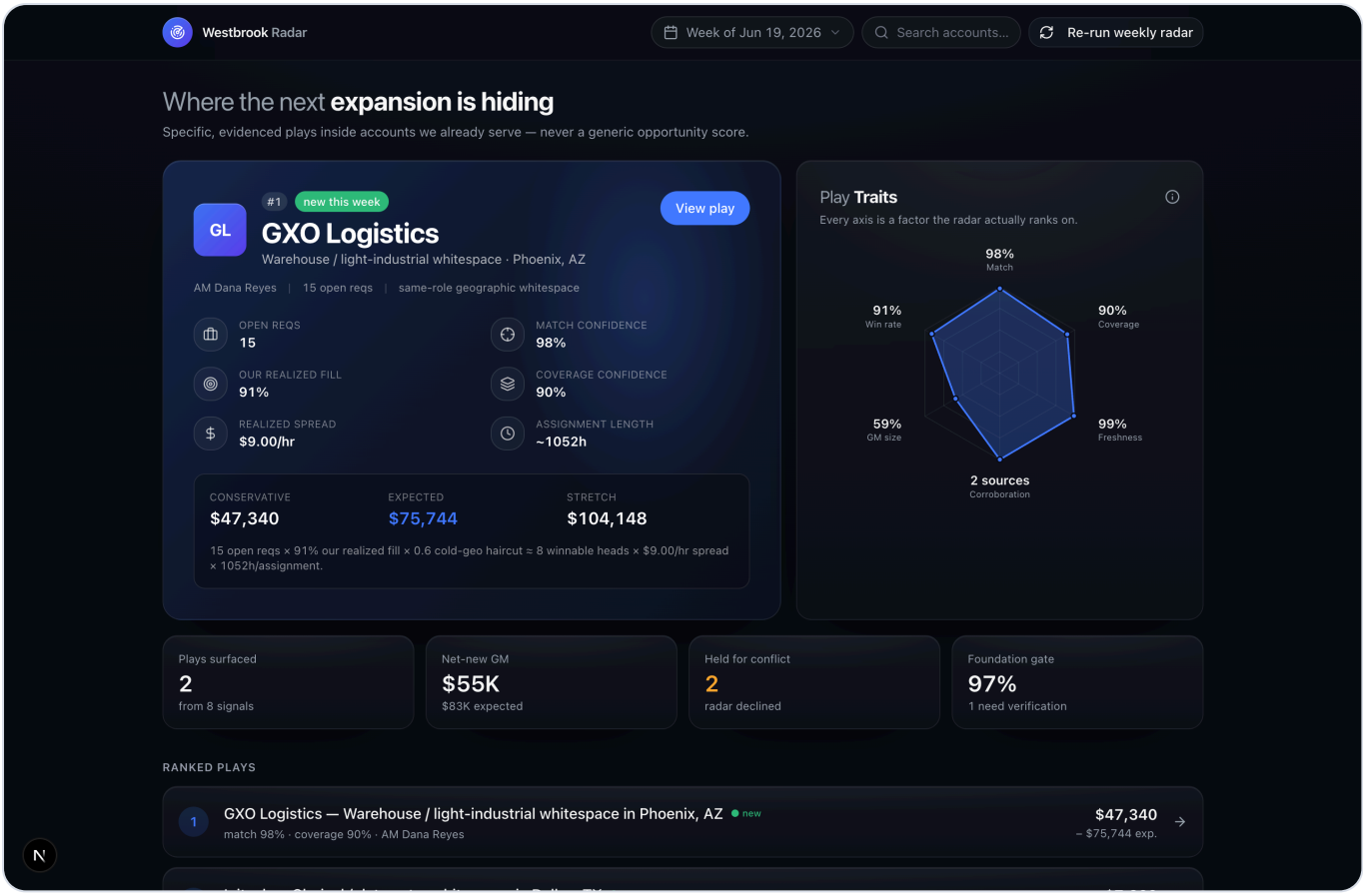
1 The reframe — what I actually built

You asked for a radar that tells account managers where the next expansion is hiding. **The radar is the easy part.** The hard part — and the real value — is two things the brief points straight at: a **trustworthy join** between your messy account history and external hiring signals, and a **guardrail** that keeps a play from ever conflicting with a client relationship.

So the first win is *not* a generic opportunity score (the brief's own trap — AMs don't trust a black-box 0–100). It is proving that **an account manager will trust and pursue one evidenced play built from your real, inconsistently-structured data — and will trust the radar to visibly hold back a play that would damage a relationship.** A system that *declines* a risky play earns more trust than ten it surfaces. Everything ladders to net-new gross margin and to the real constraint: **AM trust and adoption.**

> *"You asked for a radar. The radar is the easy part. The hard part — and the actual value — is the trustworthy join between your messy account history and external hiring signals, plus the guardrail that keeps a play from conflicting with a client relationship. That's the slice I built."*

The unit of value is an **evidenced play**, never a score. It answers the only four questions an AM has: *Can we deliver it? Is it actually open to us? What's it worth — and why should I believe the number? What could blow up the relationship?*



The working demo — the #1 play (real GXO Phoenix hiring signal joined to our Dallas placement history), a "Play Traits" hexagon plotting the actual ranking factors, a haircut gross-margin band, and — below the fold — held-for-conflict and considered-&-dropped lists.

2 Deliverable 1 — The Bottom-Up MVP ("Radar v0")

Smallest working loop: one weekly run over **5 pinned pilot accounts**, **one archetype built for real** (same-role geographic whitespace), everything else a static row or a stub. Delivered into the AM's existing weekly surface — never a new tool to remember.

What it does: pull internal facts (and time the dirt) → pull external triggers from a licensed postings API → resolve identity **closed-world** to the 5 accounts → deterministic gap detection → **grounded** GM band with visible math → evidenced play with a live citation + editable opener → rank by a transparent rule → capture the disposition (pursue | already-on-it | not-now | not-a-fit + reason).

What it deliberately ignores (each safe for now): full ATS cleanup (a *dirt log* measures the mess so the real ask is evidenced), real-time webhooks (cadence is weekly), auto-send outreach (the AM owns the send — permanently), a learned ML ranker (no training data; a black box erodes trust), perm-fee / funding / M&A signals, voice/telephony, multi-tenant/SSO, and the 2nd/3rd archetypes (static "also surfaced" rows).

How we'd know within weeks — leading vs lagging, gaming-resistant

- **Week 1 (foundation gate):** publish the real entity/role/geo resolution rate on the pilot slice. If it's low, *fix data before generating plays.*
- **Weeks 1–2 (leading):** AMs mark ≥3 of 10 as **pursue** (behavior, not a vanity click), with sensible kill-reasons — reported friendly-vs-skeptic so selection bias is visible.
- **Weeks 2–4 (safety, the primary gate):** landmine **recall** — catch dangerous false-negatives, not the cheap false-positive.

- **Weeks 3–6 (the real test):** ≥1 play reaches an actual client conversation *and* ≥1 GM estimate is reconciled against actual won spread. One account held out as a control so net-new GM is auditable.
- **Kill condition:** if AMs kill most plays as `wrong-data` / `already-filled`, the join is broken — fix the foundation before scaling accounts.

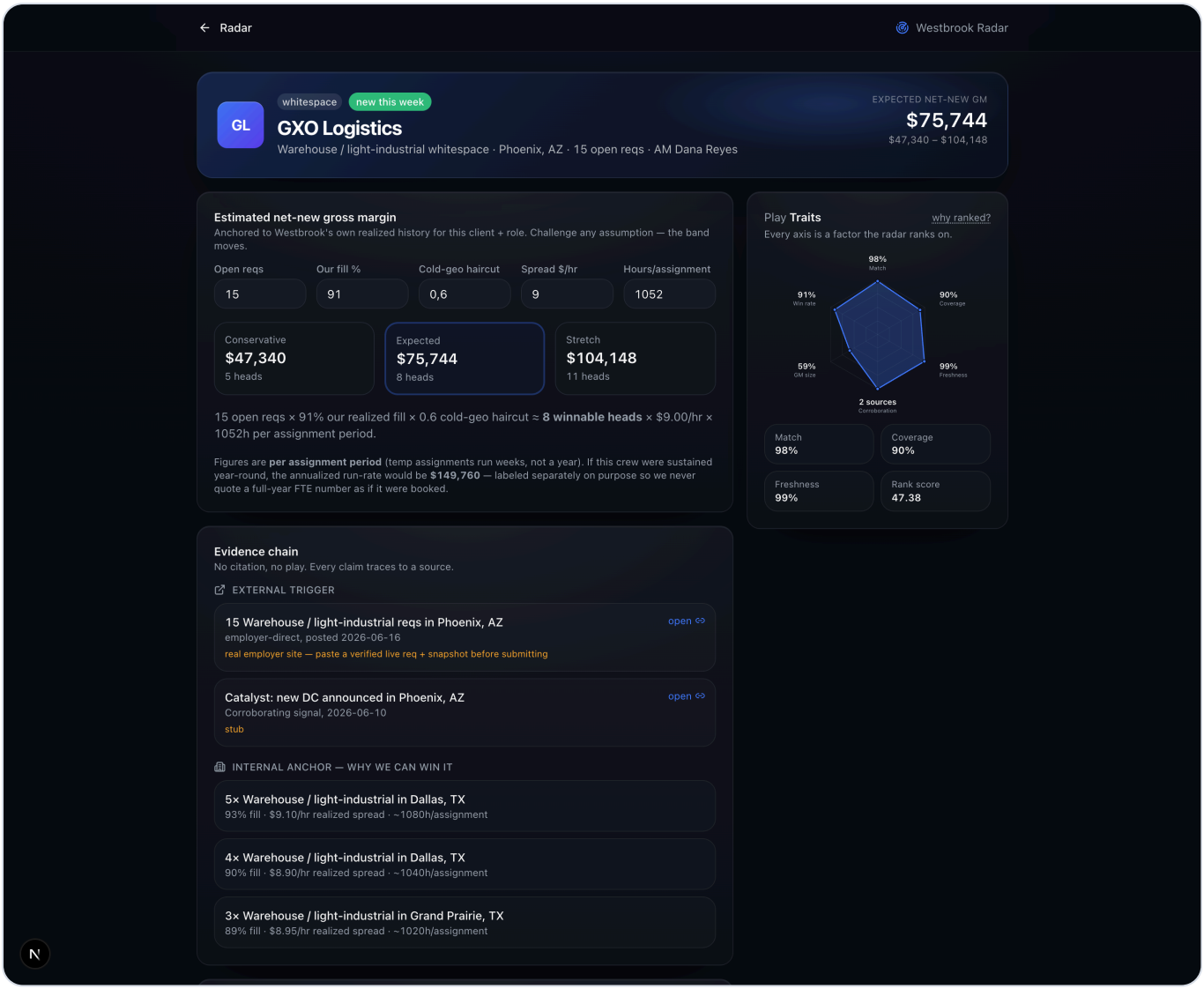
3 Deliverable 2 — System Architecture

A weekly **batch** job. Five pinned accounts, one Postgres, one screen, one run. Every box ties to net-new GM or to AM trust.

```
WEEKLY TRIGGER (Mon 6am) — external scheduler (cron / GitHub Action), idempotent run_id
        |   (NOT in-db pg_cron — >10-min ceiling, free-tier pause)
1. INGEST  -> land RAW into Postgres JSONB (the mess IS the evidence we later cite)
     internal: ATS export (placements, depts, locations, END dates, realized fill/spread/hours)
     external: licensed postings API + WARN feed + announced-DC news (employer-direct)
2. RESOLVE (closed-world -> 5 pinned ids): deterministic blocking + LLM tie-break ->
     match + confidence + reason;  below floor -> HELD for review, never silently joined
     + LLM structured extraction (messy title/geo -> typed cols, WITH confidence)
3. POSTGRES: accounts . placements . signals . play_candidates . plays . feedback . outcomes
4. GAP DETECT (SQL) -> candidate + coverage-confidence (true whitespace vs unmatched data)
   GM (grounded formula, NOT free LLM math):
       heads = open_reqs x OUR realized fill x cold-geo haircut
             x OUR realized spread x OUR realized hours  -> band (cons/exp/stretch)
   GENERATE: LLM writes narrative + editable opener
   GROUNDING GATE (two-part): every URL re-fetches live AND every GM input traces to a row -> else DROP
5. RANK (transparent): conservative_GM x match_conf x coverage_conf x freshness x corroboration
HUMAN GATE — sales-ops vets the batch before AMs see it (auto-publish is *earned*)
6. AM SURFACE (Next.js): play cards . evidence panel (live links + our placement rows) .
   GM band w/ visible math . editable never-auto-sent opener . [Pursue][Already-on-it][Not-now][Not-a-fit]
FEEDBACK + OUTCOME -> adoption signal + manual GM attribution (credit only incremental wins vs control)

[Future seams, out of MVP] pgvector . per-AM RLS . webhooks . graph DB . automated attribution . voice
```

Durable spine (v1 isn't throwaway): normalized schema · a pinned-entity match table with confidence · an **immutable evidence store that snapshots each posting at capture time** (so a play stays reproducible after the posting is deleted — a 404'd link next week destroys trust) · the play record · the disposition/feedback log. The demo implements this loop end-to-end against flat-file fixtures.



Play detail — an editable, fully-visible GM band (challenge any assumption and the band moves), the evidence chain (live external link beside our own placement rows), an LLM-drafted-but-never-auto-sent opener, and the relationship checkpoint + disposition capture.

4 Deliverable 3 — Build vs. Buy

Buy the commodity, fix the foundation, build the judgment. The raw account data isn't a moat (you own it badly; competitors own theirs). The moat is what the system **accumulates**: the cleaned join + Westbrook-specific resolution rules + the growing library of AM accept/dismiss decisions no vendor can see.

Capability	Decision	Why (cost / latency / control / lock-in)
CRM/ATS (system of record)	**Buy/keep, read-only v1**	Rebuild = multi-year, $0 GM. Write-back is a governance decision the client owns.
ATS normalization (foundation)	**Build — first, largest**	No trustworthy join until role/geo collapse to canon. Our data → our code; zero lock-in.
External job postings	**Integrate (licensed API)**	Never scrape (ToS/legal, breakage, no edge). Gate on a coverage spike-test on 5 real accounts first.
Non-posting signals (DC, WARN)	**Integrate — cheap/public**	The best staffing signal (new DC → mass hiring) isn't on a job board. v1 picks one public source.
Entity resolution	**Build thin**	Deterministic + fuzzy + LLM tie-break + conservative floor. The threshold is a relationship-risk decision.
LLM inference	**Buy usage-based**	~low-tens-of-$/run. An `LLMClient` interface → a swap is a re-test, not a no-op.
Data store	**Buy managed Postgres + JSONB**	One engine; the "graph" at MVP is relational tables. The anti-lock-in choice.
Evidenced-play generator + suppression	**Build — the reason we're here**	Output contract: specific, evidenced, citable plays — never a bare score.
Orchestration	**Buy thin — one cron**	Weekly cadence. No Temporal/Inngest (naming scale-stage tooling is range-showing).
AM surface	**Build UI, rent the rail**	Own where adoption is won; rent the Slack/Teams digest into where AMs already work.
Voice / telephony	**Don't build**	Highest cost/latency, lowest trust, $0 GM at validation. An AI calling clients is the fastest landmine.

Lock-in, honest: thin interfaces make a swap a *bounded re-integration + re-test, not a free config flip.* Cheap to **prove** (~low-hundreds–$1–2K/mo infra); the real commitments are build-time and, at scale, the enterprise data contract ($15–50K/yr). Prove net-new GM on commodity infra first, sign the data contract second.

5 Deliverable 4 — Human-in-the-Loop Ramp

Earn the right to automate; never automate the relationship. The dollar figure is risk-adjusted — a bare score is banned (unfalsifiable) and so is an un-haircut dollar figure (a score wearing a dollar sign).

- **Stage 0 — Concierge / Wizard-of-Oz (wk 0–3).** Human does more than the machine. One signal, 2–3 AMs (incl. a skeptic), ~10 accounts. *Foundation test first:* ≥90% match precision on a hand-checked sample. **Graduate:** ≥50% of plays pursued + "I'd not have spotted this"; ≥5 expansion conversations booked; ≥20 landmine reason-codes harvested.
- **Stage 1 — Assisted radar (wk 3–10).** Machine drafts, human is editor-in-chief, inside existing CRM/Slack. AM always sends. AM-owned suppression auto-hides locked/at-risk accounts. **Graduate:** landmine recall ≥90% (gated *before* the cheap false-positive); ≥60% of AMs act weekly; surfaced→pursued ≥45% and ≥1 win vs a matched control.
- **Stage 2 — Supervised semi-automation (wk 10–18).** Auto-drafts in the AM's tone; routes **green** to a ready-to-send queue, **amber** (any conflict/thin evidence/low match-conf) to mandatory review. The guardrail is *not* a bespoke ML classifier — it's AM-owned hard suppression + an LLM reviewer that cites evidence and defaults to amber. Send is always human.

- **Stage 3 — Automate the mechanical, supervise the judgment (wk 18+).** Full pipeline automation. **Never automated:** (1) the relationship judgment; (2) the client conversation.

Attribution makes the KPI falsifiable: from Stage 1, report *incremental* net-new GM (pilot – control), or audited AM "would-not-have-found" attestation where no clean control exists.

6 The working demo (the centerpiece)

A thin, real slice of the loop — the engine is deterministic code; the LLM only drafts the opener. Run it:

```
cd westbrook-radar
npm install
npm run radar     # runs the pipeline, prints a verification summary, writes data/plays.json
npm run dev       # http://localhost:3000
```

What to look at: the #1 play (a *real* GXO hiring signal joined to a realistic Dallas placement history); the haircut GM band with visible, *editable* math; the "Play Traits" hexagon (each axis is a real ranking factor); the **held-for-conflict** list (one AM-flagged hard, one surfaced from free-text notes for the AM to confirm); the **considered-&-dropped** tray with reasons (already-served, stale/ghost req, non-client-specific noise); and the **foundation gate** (how clean the join is — the metric we'd publish in week 1).

Honest demo convention: external signals are *real* (verify the links); internal placement records are a realistic stand-in until we get the ATS export. One archetype is built for real; the rest are surfaced by lighter heuristics or shown as static rows. The LLM is a formatter only — it never picks accounts, computes GM, or ranks; with no API key it falls back to a deterministic template.

7 Assumptions (stated as decisions — tell me where I'm wrong)

Read-only ATS exports in v1 (no write-back) · weekly batch cadence (matches the brief and the assignment-end-date clock) · ~hundreds of accounts → Postgres, not a distributed store · external = licensed API + WARN + DC feeds, **never scraping** · 5 pinned accounts so matching is closed-world; pilot includes a **skeptic** AM · the AM owns all client contact and the relationship-flag list · **confirm AMs are comp'd on expansion GM** (if not, that's a foundation-fix before build) · every GM input anchored to Westbrook's **own realized** history; sparse data labeled "directional" · one account held out as a control.

> *Where I assumed wrong, tell me — these are the cheapest things to change and the first I'd validate on day one.*

Confidence is in the restraint: one archetype done undeniably well, a gross-margin number deliberately discounted, a radar that declines plays, and a clear list of what I chose not to build. Boring, durable choices under the hood — Postgres + a batch job + one LLM call + a thin UI — because boring is what survives the handoff to your team.